

2

Monday
Week 49: 337-029Question 1:

(1) & (2): 18, 7, 25, 14, 3, 21, 10, 30

Iteration 1:key = 7 & $18 > 7$, move 18 right side
7, 18, 25, 14, 3, 21, 10, 30Iteration 2:key = 25, $18 < 25$, No shifting
7, 18, 25, 14, 3, 21, 10, 30Iteration 3:key = 14 & $25 > 14$, move 25 right side
 $18 > 14$, move 18 right side
 $7 < 14$, No shifting
7, 14, 18, 25, 3, 21, 10, 30Iteration 4:key = 3, $25 > 3$, Move 25 right side
 $18 > 3$, Shift right; $14 > 3$ shift; $7 > 3$ shift
3, 7, 14, 18, 25, 21, 10, 30Iteration 5:key = 21; $25 > 21$, Shift right
 $18 < 21$, No Shifting
3, 7, 14, 18, 21, 25, 10, 30Iteration 6:key = 10 & $25 > 10 \rightarrow$ Shift right
 $21 > 10 \rightarrow$ Shift; $18 > 10 \rightarrow$ Shift $\rightarrow 14 > 10 \rightarrow$ Shift
 $7 < 10 \rightarrow$ No Shifting
3, 7, 10, 14, 18, 21, 25, 30Iteration 7:key = 30 & $25 < 30 \rightarrow$ No Shifting
3, 7, 10, 14, 18, 21, 25, 30.

(3) Worst case $O(n^2)$:

Worst case occurs when array is already in descending order for each key we have shift all elements.

So, the shifting steps can be: $1+2+3+\dots+(n-1)$.

$$\text{So summation} = \frac{n(n-1)}{2}$$

$$T(n) = \frac{n^2 - n}{2}; \text{ So Time complexity} = O(n^2)$$

(4) Loop Invariant:• Initialization:

→ Before each iteration of the outer loop, elements from 0 to $i-1$ are sorted and contain the same elements that were originally in those positions.

→ Before the iteration only first element is present

• And first element is already sorted.

(Initialization True)

• Maintainance:

→ During each iteration, we take key and compare with each element larger elements shifted right, and key is inserted in correct position. So, sorted position remains sorted.

(Invariant maintained).

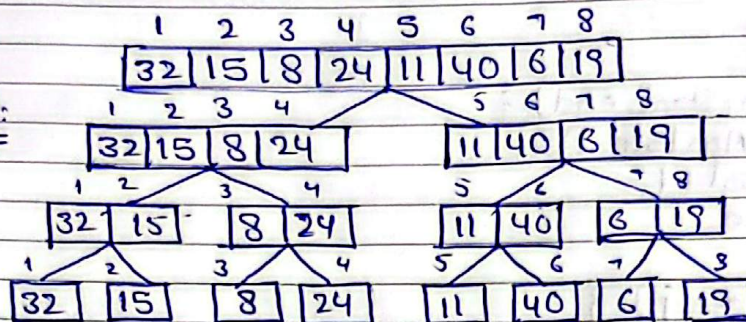
• Termination:

When the loop finishes, the sorted position contains all n elements, and array is sorted in ascending order.

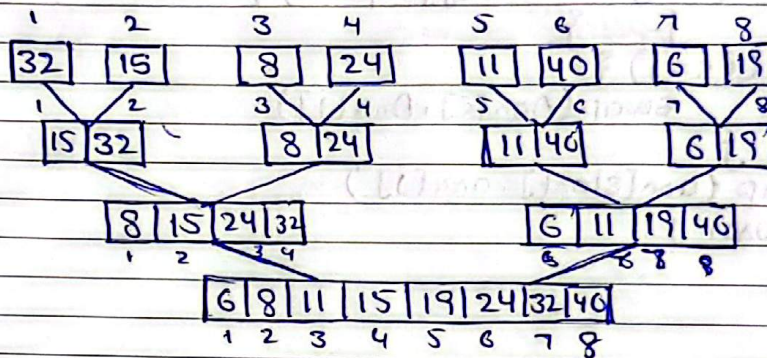
(Termination True).

Question 2

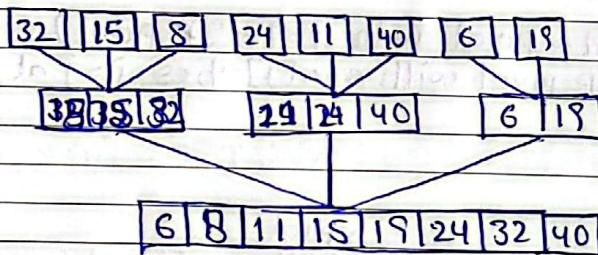
Division Method:



Recursive Sorting:



3-way merge:



$$T(n) = 3T(n/3) + n$$

Using Master's theorem

$$a=3, b=3, k=1, p=0$$

$\log_3 3 = 1$ & $\log_3 a = k$ if it is & $p > -1$
then time complexity will be $\Theta(n^k \log^{p+1} n)$

So, $\Theta(n \log n) \rightarrow$ proved.

Array
MergeSort(st, end) {
 mid1 = (st+end)/3;
 mid2 = 0
 if (st >= end) {
 fact = (end-st)/3
 mid1 = st + fact
 mid2 = st + 2*fact
 }

MergeSort(st, mid1) $\rightarrow T(n/3)$

MergeSort(mid1+1, mid2) $\rightarrow T(n/3)$

MergeSort(mid2+1, end) $\rightarrow T(n/3)$

Merge(st, mid1, mid2, end) $\rightarrow n$

Question 3

(Quick Sort):

24, 9, 17, 5, 31, 12, 27, 3
 ↓ i j
 Pivot

①

QuickSort(Ann, start, end) {

pivot = Ann[start]

i = start + 1

j = end

while (i < j) {

while (i <= j && Ann[i] <= pivot) {

i++;

while (i <= j && Ann[j] > pivot) {

j--;

if (i <= j) {

swap(Ann[i], Ann[j]);

}

swap(Ann[start], Ann[j]);

return j;

}

②

Partition 1:

0 1 2 3 4 5 6 7

• 24, 9, 17, 5, 31, 12, 27, 31

pivot = 24, i = 9, j = 31

• 24, 9, 17, 5, 31, 12, 27, 31 (Swap 31 > pivot with j = 7 ⇒ 31 < pivot)

• 12, 9, 17, 5, 3, 24, 27, 31 (Swap pivot with Ann[j] bcs i > j at j = 5 & i = 6)

Partition 2:

12, 9, 17, 5, 3, 24, 27, 31

pivot = 12, i = 9, j = 31

12, 9, 3, 5, 17, 24, 27, 31 (Swap Ann[i] at i = 2 with Ann[j] at j = 4 < pivot)

5, 9, 3, 12, 17, 24, 27, 31 (Swap pivot with Ann[j] at j = 3; i > j)

Partition 3:

5, 9, 3, 12, 17, 24, 27, 31

pivot = 5, i = 9, j = 31

5, 3, 9, 12, 17, 24, 27, 31 (Swap Ann[i] with Ann[j] at i = 1 & j = 2)

3, 5, 9, 12, 17, 24, 27, 31 (Swap pivot with Ann[j] at j = 1; i = 2 > j = 1)

18 Wednesday
Week 51 - 353-013③ loop invariant

• Initialization:

 $\text{start} = 0, \text{end} = \text{sizeof}(\text{arr}) - 1$ $i = \text{start} + 1, j = \text{end}$ $A[\text{start} + 1 \dots i - 1]$ is empty $A[j + 1 \dots \text{end}]$ is empty

Invariant proved because at right and left it fulfills conditions.

• Maintenance

→ While ($A[i] \leq \text{pivot}$): $i++$ every element is $\text{arr}[i] \leq \text{pivot}$ then $\text{arr}[\text{start} + 1 \dots i - 1] \rightarrow$ left side→ While ($A[j] > \text{pivot}$): $j--$ every element is $\text{arr}[j] > \text{arr}[\text{pivot}]$ then $\text{arr}[j + 1 \dots \text{end}] \rightarrow$ Right side⇒ if ($i < j$) $\text{arr}[i] > \text{pivot} \ \& \ \text{arr}[j] < \text{pivot}$ swap $\text{arr}[i]$ with $\text{arr}[j]$ means

smaller element moves towards left and right towards is longer.

invariance proved.

• Termination

while ($i > j$)swap ($\text{arr}[\text{start}], \text{arr}[j]$)

Doing pivot keeps its correct position

So termination proved.

Question 4

For dividing the skill rating into two teams, so to divide two we divide rating sum into smaller and large, means each team has ^{either} smaller or larger sum.

① $O(n^2)$

SumA = 0, SumB = 0

for this firstly use nested loop to find max and add the max with sum that is less than other like (SumA < SumB) add max in sumA otherwise Sum B.

```
for (i=0; i<2*n; i++) {  $\rightarrow n+1$ 
```

```
    max = -1, index = -1;
```

```
    for (j=0; j<2*n; j++) {  $n \times n$ 
```

```
        if (arr[j] > max) {  $\rightarrow 1$ 
```

```
            max = arr[j];  $\rightarrow 1$ 
```

```
        }
```

```
    if (sumA <= sumB && countA < n) {  $\rightarrow 1$ 
```

```
        sumA += max;  $\rightarrow$ 
```

```
        countA++;
```

```
    } else {
```

```
        sumB += max;
```

```
        countB++;
```

```
    }  $O(n^2)$ 
```

② $O(n \log n)$

First sort the all element in increasing order using merge Sort and it takes time complexity of $O(n \log n)$ and the assignment time complexity will be $O(n)$

Sort (Arr) $\rightarrow O(n \log n)$

```
for (i=0; i<n; i++) {  $\rightarrow O(n)$ 
```

```
    if (sumA <= sumB) {
```

```
        sumA += arr[i];
```

```
    } else {
```

```
        sumB += arr[i];
```

```
    }
```

$T(n) = n \log n + n$

$O(n \log n)$.

Question 5:

2, 4, 6, 8, 10, 12

$$2n = 6$$

$$n = 3 \rightarrow \text{pairs}$$

1): To minimize the largest pair sum, firstly pair small element with largest element

e.g: $(12, 2) = 12 + 2 \Rightarrow 14$

$(10, 4) = 10 + 4 \Rightarrow 14$

$(6, 8) = 6 + 8 \Rightarrow 14$

Since all sums are 14, So
maximum pair sum = 14.

2) For $O(n \log n)$ time complexity, we should first sort the array in ascending order and then pair up smallest element with largest element.

Pairing Numbers (Ans) {

$$\text{MergeSort(Ans)} \implies O(n \log n)$$

$$\text{left} = 0, \text{right} = \text{Ans.length}() - 1$$

$$\text{MaxSum} = 0$$

$$\text{while (left < right) } \{ \implies O(n)$$

$$\text{sum} = \text{Ans}[\text{left}] + \text{Ans}[\text{right}]$$

$$\text{if (sum > MaxSum) } \{$$

$$\text{MaxSum} = \text{sum}$$

$$\}$$

$$\text{left}++$$

$$\text{right}--$$

}

$$T_n = O(n \log n) + O(n)$$

$$TC = O(n \log n) \rightarrow \text{Proved.}$$

16

Monday
Week 51 · 351-015Question 6

$$(a): n^2 + 5n + 10 = O(n^2)$$

$$n^2 + 5n + 10 \leq cn^2$$

$$n^2 + 5n + 10 \leq n^2 + 5n^2 + 10n^2$$

$$n^2 + 5n + 10 \leq 16n^2$$

$$c = 16, n_0 = 1$$

$$(b): 4n^2 \log_2 n + 3n^2 = O(n^2 \log_2 n)$$

$$4n^2 \log_2 n + 3n^2 \leq cn^2 \log_2 n$$

$$4n^2 \log_2 n + 3n^2 \leq 4n^2 \log_2 n + 3n^2 \log_2 n$$

$$4n^2 \log_2 n + 3n^2 \leq 7n^2 \log_2 n$$

$$c = 7, n_0 = 2$$

Question 7

1): Asymptotic notation in algorithm analysis tells the how grows time complexity and space complexity as increasing the value of n and also to identify the efficiency of algorithm.

2):

Big-O: Denotes the upper bound of an algorithm's growth rate, the maximum growth of an algorithm.

Big- Ω : Denotes the lower bound of an algorithm's growth rate, the minimum growth of an algorithm.

Big- Θ : Denotes the tight bound of an algorithm. the growth rate of an algorithm will be as same as upper bound and lower bound growth rate.

3):

For constants value there is larger leading term on increasing the value of n the greater term grows larger so growth rate increases that's constants and lower order terms generally ignored.

4): For single loop time complexity will be linear $O(n)$.

For nested loop time complexity will be quadratic $O(n^2)$ as input grows $O(n^2)$ grows rapidly and $O(n)$ is efficient.

(5):

14 Saturday
Week 50 · 349-017Question 8:

1): $T(n) = \sqrt{2} T(n/2) + \log n$

$a = \sqrt{2}, b = 2, k = 0, p = 1 \text{ \& } f(n) = \log n$

$\log_b a \Rightarrow \log_2 (2)^{1/2} = 1/2 \text{ on } 0.5$

it means;

$\log_b a > k \Rightarrow 0.5 > 0$

if $\log_b a > k$ then $TC = O(n^{\log_b a})$

So,

$O(n^{\log_2 (2)^{1/2}}) \text{ on } O(\sqrt{n})$

(2): $T(n) = 64 T(n/8) - n^2 \log n$

$a = 64, b = 8, k = 2, p = 1 \text{ \& } f(n) = -n^2 \log n$

$\log_b a \Rightarrow \log_8 (2)^6 = 2$

$\log_8 64 = k$: it means if $\log_b a = k$ then TC depends on p
if $p > -1$ $TC = O(n^k \log^{p+1} n)$

So,

$TC = O(n^2 \log^2 n)$

(3): $T(n) = 16 T(n/4) + n!$

$a = 16, b = 4, f(n) = n!$

$\log_4 16 \Rightarrow 2$

and there is $f(n) = n!$ and it grows more and more so leading term or will be larger of $f(n)$ so

$TC = O(n!)$

Question 9

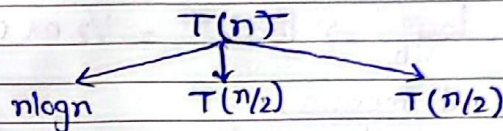
$$(1): T(n) = \sqrt{2} T(n/2) + n \log n$$

$$T(n) = 2 T(n/2) + n \log n$$

$$T(n) = 2^2 T(n/2^2) + n/2 \log(n/2) + n \log n$$

$$T(n) = 2^3 T(n/2^3) + n/2^2 \log(n/2^2) + n/2 \log(n/2) + n \log n$$

$$T(n) = 2^k T(n/2^k) + (n/2^{k-1} \log(n/2^{k-1}) + \dots + n \log n)$$



$$T(n/2) = 2 T(n/2^2) + n/2 \log(n/2)$$

$$T(n) = 2 (2 T(n/2^2) + n/2 \log(n/2)) + n \log n$$

$$T(n) = 2^2 T(n/2^2) + n \log(n/2) + n \log n$$

$$T(n) = 2^k T(n/2^k) + (n \log(n/2^{k-1}) + \dots + n \log n)$$

$$= 2^k T(n/2^k) + n (\log(n/2^{k-1}) + \log(n/2^{k-2}) + \dots + \log n)$$

$$= 2^k$$

$$\frac{n}{2^k} = 1 \Rightarrow n = 2^k$$

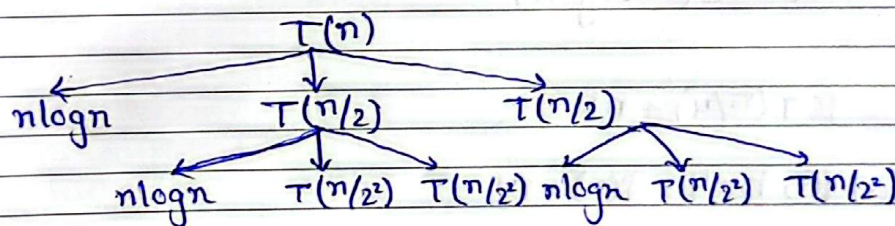
$$k = \log_2 n$$

$$= n T(1) + n (\log^2 n)$$

$$T(n) = O(n) + O(n \log^2 n)$$

So,

$$TC = O(n \log^2 n)$$



DECEMBER

12 Thursday
Week 50 · 347-019

$$8^k T(n/2^k) + (n/2^{k-1})^2$$

$$\frac{8^3}{2^4} = \frac{2^1}{2^3} = \frac{2^6}{2^4} = \frac{2^2}{2^4} = \frac{2^2}{2^4}$$

$$(2): T(n) = 8T(n/2) + n^2$$

$$T(n/2) = 8T(n/2^2) + (n/2)^2$$

$$T(n) = 8^2 T(n/2^2) + (n/2)^2 + n^2 \Rightarrow 8^2 T(n/2^2) + 8(n/2)^2 + n^2$$

$$T(n/2^2) = 8T(n/2^3) + (n/2^2)^2$$

$$T(n) = 8^3 T(n/2^3) + (n/2^2)^2 + (n/2)^2 + n^2 \Rightarrow 8^3 T(n/2^3) + n^2/8 + n^2/2 + n^2$$

$$T(n) = 8^k T(n/2^k) + (n/2^{k-1})^2 \Rightarrow 8^k T(n/2^k) + \sum_{k=1}^n (n/2^{k-1})^2$$

$$T(n) = 8^3 T(n/2^3) + 4n^2 + 2n^2 + n^2$$

$$T(n/8) = 8^3 T(n/2^4) + 4(n/2^3)^2 + 2(n/2^3)^2 + (n/2^3)^2$$

$$T(n) = 8^4 T(n/2^4) + 32(n/64) + 16(n/64)$$

$$T(n/2^3) = 8^3 T(n/2^4) + (n/2^3)^2$$

$$T(n) = 8^4 T(n/2^4) + 8^3 (n/2^3)^2 + 4n^2 + 2n^2 + n^2$$

$$= 8^4 T(n/2^4) + 8n^2 + 4n^2 + 2n^2 + n^2$$

$$T(n) = 8^k T(n/2^k) + 2^{k-1} n^2$$

$$n/2^k = 1 \Rightarrow n = 2^k \Rightarrow k = \log_2 n$$

$$2^{3k} \Rightarrow (2^k)^3$$

$$\text{So, } 2^k = n$$

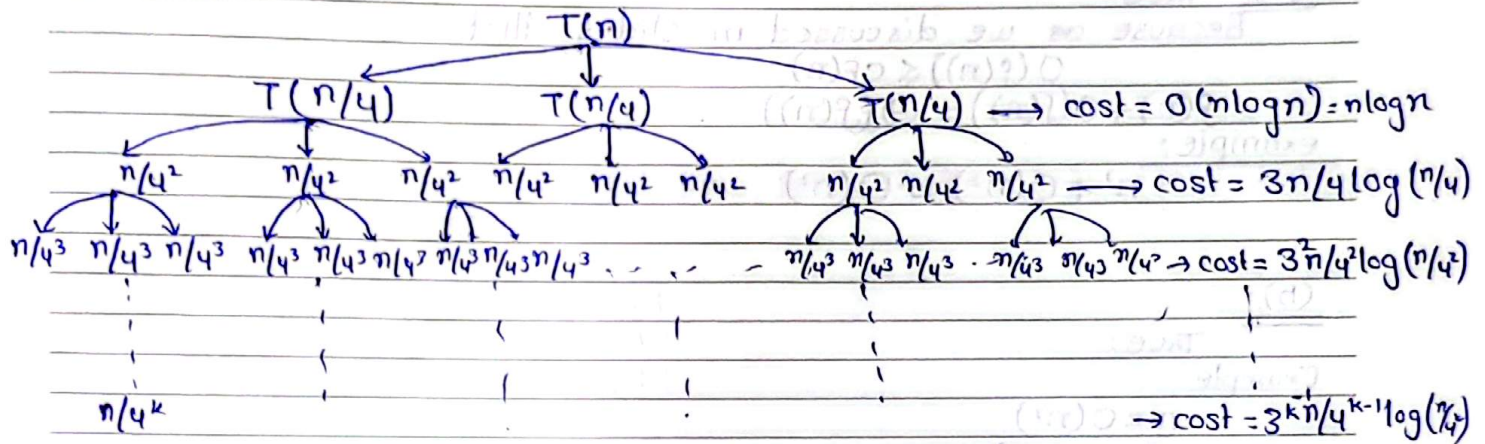
$$(n)^3 \Rightarrow n^3$$

$$T(n) = n^3 T(1) + \frac{2^k}{2} n^2 \Rightarrow \frac{n^3 + n(n^2)}{2} \Rightarrow \frac{n^3 + n^3}{2}$$

$$TC = O(n^3)$$

Question 11:

(a). $T(n) = 3T(n/4) + n \log n$



$$\begin{aligned} n/4^k &= 1 \\ n &= 4^k \\ k &= \log_4 n \end{aligned}$$

$$\begin{aligned} \text{Total cost} &= n \log n + 3n/4 \log(n/4) + 3^2 n/4^2 \log(n/4^2) + \dots + 3^{k-1} n/4^{k-1} \log(n/4^{k-1}) \\ &= n (\log n + 3/4 \log(n/4) + \dots + 3^{k-1}/4^{k-1} \log(n/4^{k-1})) \end{aligned}$$

$$= n (\log n + 3/4 (\log n - \log 4) + \dots + 3^{k-1}/4^{k-1} (\log n - \log(4^{k-1})))$$

$$= n$$

$$= n [\log n (1 + 3/4 + 3^2/4^2 + \dots) - (3/4 + 3^2/4^2 + 3(3^3/4^3) + \dots)]$$

$$= n [\log n (1/(1-3/4)) - \log 4 (3/4 + 2(3^2/4^2) + 3(3^3/4^3) + \dots)]$$

$$= n [\log(n) \times 4 - \log 4 (3/4 / (1-3/4)^2)]$$

$$= 4n \log(n) - \frac{3/4 \log(4)}{(1-3/4)^2} \Rightarrow 4n \log_4(n) - 12n$$

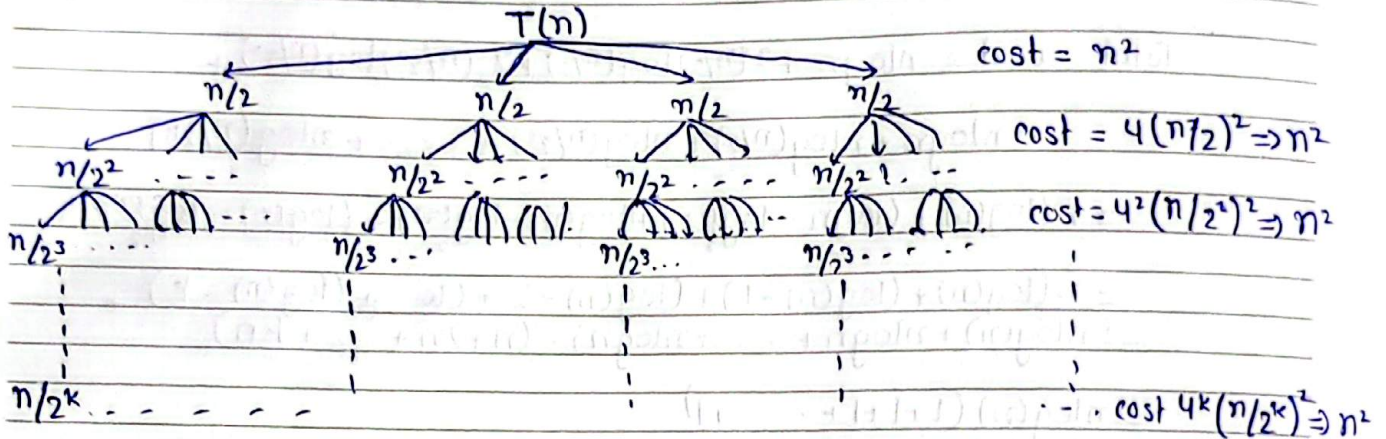
$$T(n) = \Theta(n \log_4(n))$$

30 Saturday
Week 48 - 335-031

Question (11)

(b):

$$T(n) = 4T(n/2) + n^2; \quad T(1) = \Theta(1)$$



$$n/2^k = 1$$

$$n = 2^k \Rightarrow k = \log_2 n$$

$$\text{Total cost} = n^2 + n^2 + n^2 + \dots$$

$$\text{So, } T(n) = \Theta(n^2)$$

Because continuously same n^2 is distributed among tree,
So it represents tight asymptotic bound.

Question 12):

(a): $T(n) = 2T(n/2) + n \log n$, $T(1) = \Theta(1)$

By recurrence relation

$$\text{Total cost} = n \log n + 2(n/2) \log(n/2) + 2^2(n/2^2) \log(n/2^2) + \dots$$

$$= n \log n + n \log(n/2) + n \log(n/2^2) + \dots + n \log(n/2^k)$$

$$= n(\log(n) + (\log(n) - \log_2 2) + (\log(n) - \log_2 4) + \dots + (\log(n) - \log_2 2^k))$$

$$= n(\log(n) + (\log(n) - 1) + (\log(n) - 2) + \dots + (\log(n) - k))$$

$$= (n \log(n) + n \log(n) + \dots + n \log(n)) - (n + 2n + \dots + kn)$$

$$= n \log(n) (1 + 1 + 1 + \dots + 1)$$

$$= n \log(n) \times k$$

$$n/2^k = 1$$

$$n = 2^k \Rightarrow k = \log_2 n$$

$$\Rightarrow n \log_2 n \times \log_2 n$$

$$= n \log^2 n$$

$$\bullet T(n) \leq cn \log^2(n)$$

$$T(n/2) \leq cn/2 \log^2(n/2)$$

$$\bullet T(n) \leq 2cn \log^2(n/2) + n \log(n)$$

$$\bullet T(n) \leq cn(\log(n) - 1)^2 + n \log(n)$$

$$T(n) \leq cn(\log^2(n) - 2\log(n) + 1) + n \log n$$

$$T(n) \leq cn \log^2(n) - 2cn \log(n) + cn + n \log n$$

Since here leading term is $cn \log^2(n)$

Hence Proved: $\Theta(n \log^2(n))$.

28 Thursday Thanksgiving Day (US)
Week 48 · 333-033

(b):

$$T(n) = 2T(n/4) + \sqrt{n}$$

$$\text{Total cost} = \sqrt{n} + 2(\sqrt{n/4}) + 2^2(\sqrt{n/4^2}) + \dots$$

$$= \sqrt{n} + \sqrt{n} + \sqrt{n} + \dots$$

$$= \sqrt{n}(1 + 1 + 1 + \dots) \rightarrow \text{upto } k \text{ terms}$$

$$= \sqrt{n} \times k$$

$$\begin{aligned} \bullet \quad n/4^k &= 1 \Rightarrow n = 4^k \\ k &= \log_4(n) \end{aligned}$$

$$\therefore \text{Total cost} = \sqrt{n} \log_4 n$$

$$T(n) \leq c\sqrt{n} \log n$$

$$\bullet \quad T(n) \leq 2T(n/4) + \sqrt{n}$$

$$T(n/4) \leq c\sqrt{n/4} \log(n/4)$$

$$\begin{aligned} \bullet \quad T(n) &\leq c\sqrt{n} \log(n/4) + \sqrt{n} \\ T(n) &\leq c\sqrt{n} (\log(n) - \log(4)) + \sqrt{n} \\ T(n) &\leq c\sqrt{n} \log(n) - c\sqrt{n} \cdot 2 + \sqrt{n} \\ T(n) &\leq c\sqrt{n} \log(n) - 2c\sqrt{n} + \sqrt{n} \\ T(n) &\leq c\sqrt{n} \log(n) + \sqrt{n} (2c + 1) \end{aligned}$$

Since here leading term is $\sqrt{n} \log(n)$

So, Hence Proved: $\Theta(\sqrt{n} \log(n))$

Question 10

(a): True

Because as we discussed in slides that

$$O(f(n)) \leq cf(n)$$

$$f(n) + O(f(n)) = \Theta(f(n))$$

example:

$$n^2 + O(n^2) = \Theta(n^2)$$

(b):

True:

Example:

$$n = O(n^2)$$

$$n^2 = O(n^3)$$

$$n = O(n^3)$$

(c)

True

Example:

$$f(n) = n^2$$

$$g(n) = 5n^2$$

$$f(n) + g(n)$$

$$n^2 + 5n^2 = 7n^2$$

$$7n^2 = \Theta(n^2)$$

(d) $n^2 = O(n^3)$

True

Since n^3 grows faster than n^2 (e): $n^3 = O(n^2)$

False

 n^2 not grows faster than n^3

So, it doesn't fulfill Big O notation.